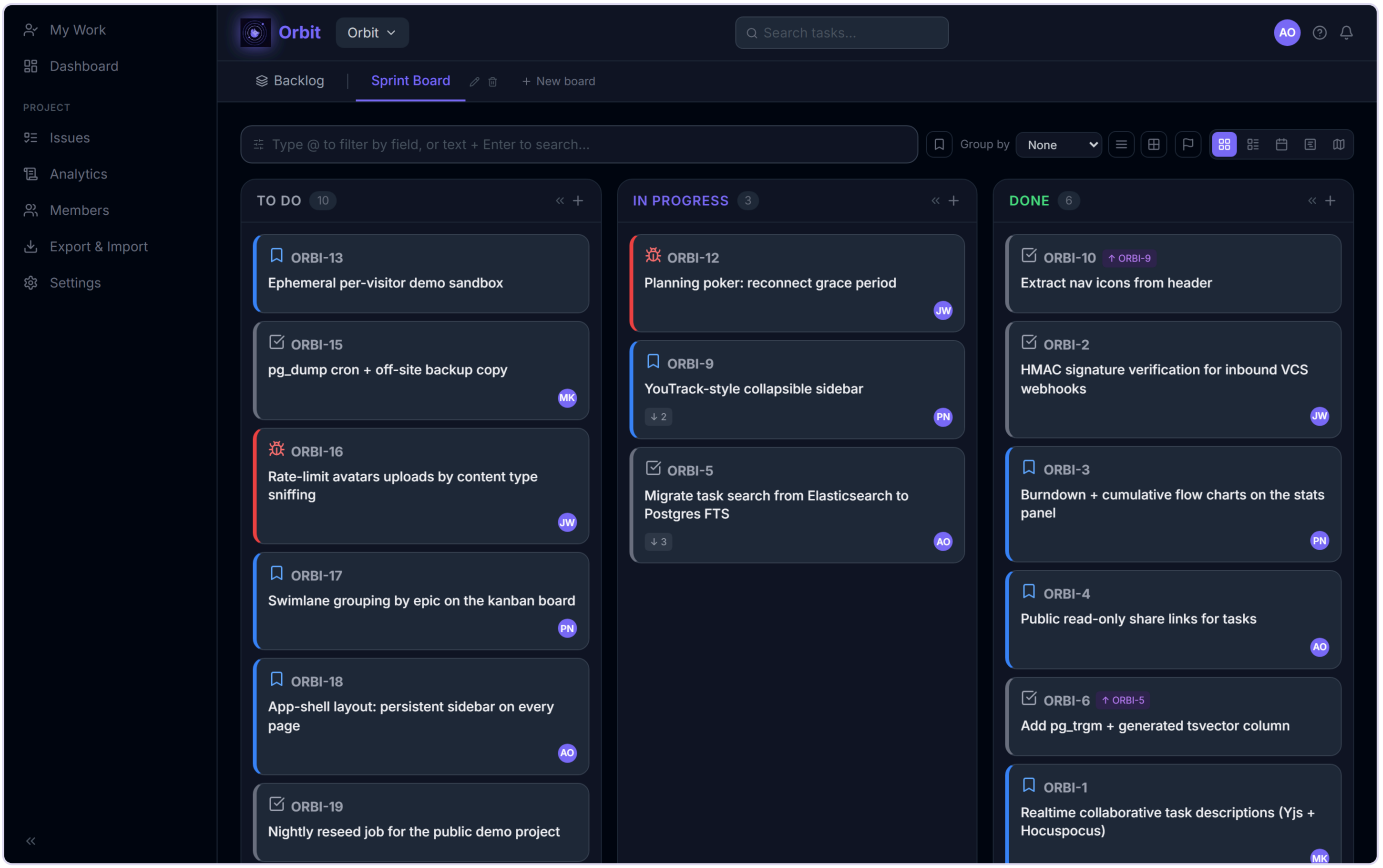
Orbit — self-hosted Kanban task manager

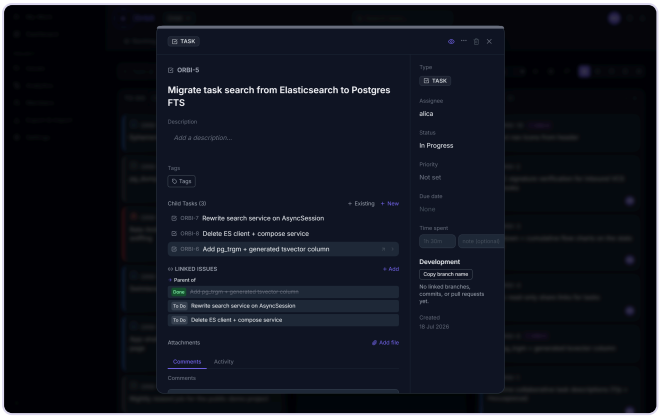
Real-time collaborative issue tracker for engineering teams. YouTrack-style power, Linear-inspired UX, and you run it on your own hardware.

FastAPI + React + Yjs collaborative editing. One `docker compose up` and it's running.

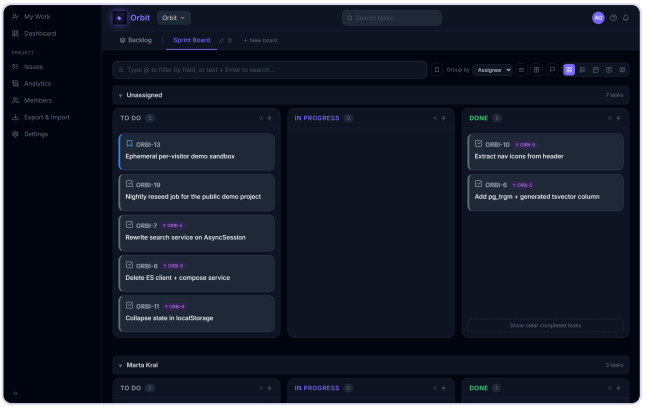


Screenshots

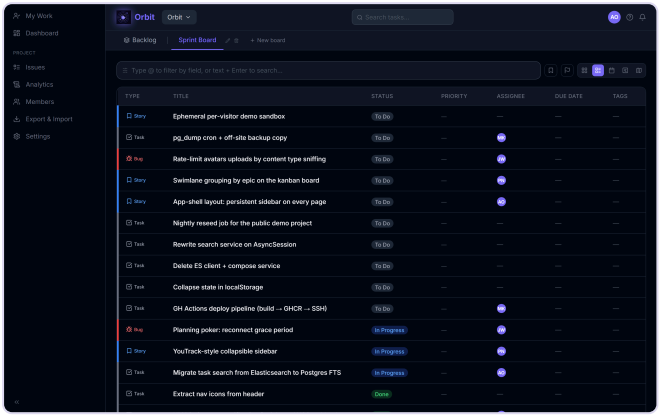
Task detail — collaborative description, comments, activity, linked branches



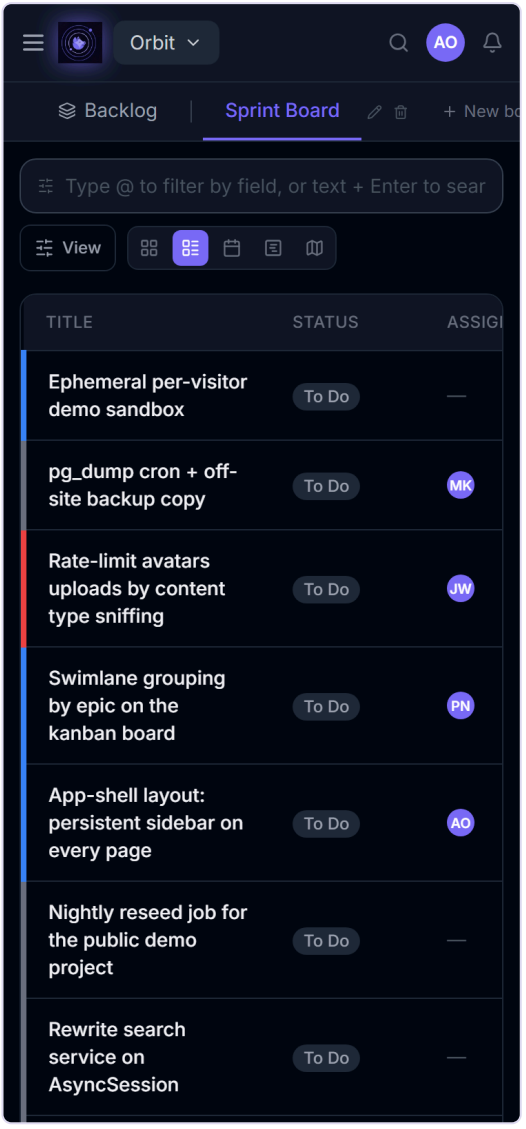
Swimlanes — group the board by assignee, priority, type or epic



List view — sortable, inline-editable table



Mobile — responsive down to 375px



Features

Boards & workflow

- Kanban board with drag-and-drop (dnd-kit); press-and-hold to drag on touch
- Five views — Kanban, List, Calendar, Gantt, Roadmap
- Swimlanes — group by assignee, priority, type or epic
- Three project modes — **Flow** (3 fixed statuses), **Guided** (custom statuses), **Enforced** (+ transition rules)
- Sprints, backlog, story points, velocity and burndown
- Task hierarchy (epics → stories → tasks), issue links (blocks / depends on / duplicates / relates to)

Collaboration

- Concurrent description editing via Yjs CRDT (Hocuspocus) — no lost work
- Comments with edit history, emoji reactions and attachments
- Watchers, in-app notifications, due-date reminders
- Planning poker, @-mentions, real-time board sync over WebSocket

Data & integration

- Git integration — GitHub / GitLab / Bitbucket webhooks link commits, branches and PRs to tasks, and can drive status transitions
- Full-text search — PostgreSQL FTS with English stemming, relevance ranking and `pg_trgm` typo tolerance (no external search service)
- `@field:value` filter language with OR/NOT/AND, plus saved filters
- CSV / Trello / Jira import, CSV export, public read-only share links
- Custom fields, task templates, automation rules, releases, recurring tasks, work logs
- REST API tokens (PAT) and outbound webhooks (JSON / Slack / Discord)

Admin & ops

- RBAC — admin / member / viewer per project; email invites
- Google SSO, email verification, password reset
- Analytics — burndown, cumulative flow, cycle time, time-in-status, audit log
- Structured logging (structlog) + OpenTelemetry traces

Stack

Layer	Tech
Backend	FastAPI, SQLAlchemy 2.0 async (asyncpg), PostgreSQL 16
Auth	PyJWT, Argon2id, Redis token blacklist
Real-time	FastAPI WebSockets, Hocuspocus / Yjs (CRDT)
Search	PostgreSQL FTS (tsvector + pg_trgm)
Storage	MinIO (S3-compatible)
Frontend	React 18, TypeScript, Vite, Tailwind CSS, TanStack Query v5
UI	dnd-kit, Tiptap v3 + CodeMirror 6, Recharts, lucide-react
Email	SMTP or Brevo API
AI (optional)	OpenAI structured outputs — subtask generation
Infra	Docker Compose (single server), Helm chart (Kubernetes)

Prerequisites

- **Docker Desktop** — runs everything with one command
- Or, for local dev: Python 3.12 + Node 20 + PostgreSQL 16 + Redis 7 + MinIO

Quick start

```
git clone https://github.com/elis4265/saas.git orbit
cd orbit
docker compose up --build
```

That's it — the backend runs `alembic upgrade head` on boot, so the schema is created for you.

URL	Service
http://localhost:3000	Frontend
http://localhost:8000	Backend API
http://localhost:8000/docs	Swagger UI
http://localhost:9001	MinIO console

Data persists in Docker volumes (`postgres_data` , `minio_data`).

> **Email in development.** Out of the box no mail is sent — registration codes are only logged. To capture them in a UI, uncomment the `mailhog` service (and the backend's `depends_on` entry) in `docker-compose.yml`, leave `SMTP_HOST` unset, and open http://localhost:8025. To send real mail, set `BREVO_API_KEY` or real `SMTP_*` values.

Useful commands

```
docker compose up --build                  # rebuild + start
docker compose build --no-cache frontend   # force-rebuild one service
docker compose down                        # stop (volumes survive)
docker compose down -v                     # stop + wipe data
docker compose logs -f backend             # follow logs
```

Local dev (faster iteration)

Backend:

```
cd taskflow-backend
pip install -r requirements.txt
alembic upgrade head
uvicorn app.main:app --reload      # http://localhost:8000
```

Frontend:

```
cd taskflow-frontend
npm install
npm run dev                        # http://localhost:5173
```

Local dev still needs Postgres, Redis and MinIO — easiest is `docker compose up db redis minio`.

Self-hosting in production

`docker-compose.prod.yml` runs the full stack behind Caddy with automatic HTTPS. Only Caddy publishes ports; everything else stays on the internal network.

```
cp .env.production.example .env    # then fill in the secrets
docker compose -f docker-compose.prod.yml up -d --build
```

Set `ORBIT_DOMAIN` to your domain, point its DNS at the server, and Caddy obtains a Let's Encrypt certificate on first boot. Generate secrets with `openssl rand -hex 32`.

> Run the backend with a **single replica**. WebSocket rooms and planning-poker state are in-process; horizontal scaling needs Redis pub/sub fanout (not implemented yet).

Kubernetes (Helm)

```
helm dependency update helm/taskflow
helm install orbit helm/taskflow \
  --set secrets.jwtSecretKey=$(openssl rand -hex 32) \
  --set secrets.hocuspocusInternalKey=$(openssl rand -hex 16) \
  --set ingress.host=orbit.yourdomain.com
```

The frontend calls the API on a relative path, so the images need no per-host build arguments. Bundled subcharts are `bitnami/postgresql`, `bitnami/redis` and `bitnami/minio` — note that Bitnami restructured its public catalog in 2025, so you may prefer to point the chart at your own managed Postgres/Redis/S3 instead.

Environment variables

Set via `taskflow-backend/.env` (local dev), the root `.env` (Docker Compose), or Helm values.

Variable	Description
`DATABASE_URL`	`postgresql+asyncpg://user:pass@host:5432/db`
`JWT_SECRET_KEY`	Long random string — `openssl rand -hex 32`
`REDIS_URL`	`redis://localhost:6379/0`
`ALLOWED_ORIGINS`	JSON array: `["https://orbit.example.com"]`
`ENVIRONMENT`	`development` or `production` (controls secure cookies and dev-only endpoints)
`BASE_URL` / `FRONTEND_URL`	Public URLs used in emails and avatar links
`MINIO_ENDPOINT`	`minio:9000` (host:port, no scheme)
`MINIO_ACCESS_KEY` / `MINIO_SECRET_KEY`	MinIO credentials
`MINIO_BUCKET`	`taskflow-attachments`
`HOCUSPOCUS_INTERNAL_KEY`	Shared secret between backend and Hocuspocus
`SMTP_HOST` / `SMTP_PORT` / `SMTP_USER` / `SMTP_PASSWORD`	SMTP email (takes priority when `SMTP_HOST` is set)
`BREVO_API_KEY`	Brevo transactional email (used when SMTP is unset)
`EMAIL_FROM`	Sender address — must be verified with your mail provider
`OPENAI_API_KEY`	Optional — AI subtask generation
`GOOGLE_OAUTH_CLIENT_ID`	Optional — Google SSO; unset hides the button
`GITHUB_OAUTH_CLIENT_ID` / `GITLAB_OAUTH_CLIENT_ID` / `GITLAB_BASE_URL`	Optional — git integration OAuth
`ENCRYPTION_KEY`	Optional — Fernet key encrypting stored VCS tokens; unset degrades git features gracefully
`ORBIT_SUPERUSER_EMAIL`	Optional — promoted to instance superuser at startup; unset means no `/admin` surface
`WEBHOOK_ALLOW_PRIVATE_IPS`	`false` — set `true` to allow outbound webhooks to private IPs

`ALLOWED_ORIGINS` must be a valid JSON array — pydantic-settings parses it as JSON.

Tests

Backend (~664 unit tests, no DB required):

```
cd taskflow-backend
pytest tests/unit -v
pytest tests/unit --cov=app --cov-fail-under=80
pytest tests/integration -v          # Testcontainers spins up ephemeral PostgreSQL
USE_TESTCONTAINERS=0 pytest tests/integration -v   # use DATABASE_URL instead
```

Frontend (~704 unit tests, no backend required):

```
cd taskflow-frontend
npm test
npx playwright install chromium && npm run test:e2e   # E2E needs the Docker stack
```

API

Interactive docs at `http://localhost:8000/docs`. Routes are project-scoped:

Method	Path	Description
POST	`/api/v1/auth/register`	Create account (verification code emailed)
POST	`/api/v1/auth/login`	Login → JWT access token + refresh cookie
GET	`/api/v1/projects`	List projects (owned + joined)
GET	`/api/v1/projects/{id}/tasks`	All project tasks
POST	`/api/v1/projects/{id}/boards/{board_id}/tasks`	Create task
PATCH	`/api/v1/projects/{id}/tasks/{task_id}`	Update task (send `version` — OCC)
GET	`/api/v1/projects/{id}/tasks/search?q=`	Full-text search
GET	`/api/v1/search/tasks?q=`	Cross-project search
GET	`/api/v1/projects/{id}/stats`	Task stats
GET	`/api/v1/notifications`	In-app notifications
WS	`/api/v1/ws/projects/{id}`	Real-time board events

Task updates use optimistic concurrency — send the task's current `version`; a `409` means someone else updated it first.

Authenticate with `Authorization: Bearer <jwt>` or a personal access token (`orbit_pat_…`).

Troubleshooting

Port 5432 already in use — a local PostgreSQL is running:

```
docker compose down && docker compose up --build
```

Tables missing after pulling changes — stale volume:

```
docker compose down -v && docker compose up --build
```

Frontend changes not appearing — Docker layer cache:

```
docker compose build --no-cache frontend && docker compose up
```

Alembic version mismatch (after integration tests wipe `alembic_version`):

```
alembic stamp base && alembic upgrade head
```

License

See LICENSE.